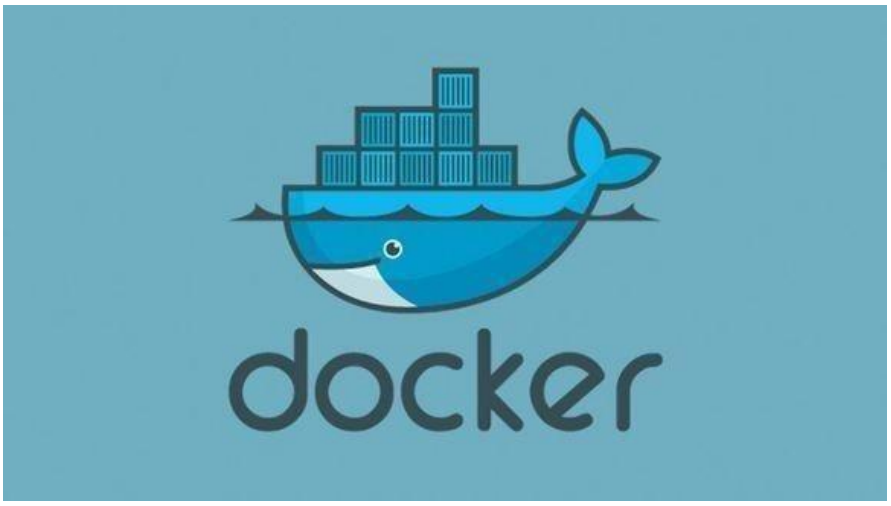
Experiment-9: Working with Docker

Aim: To demonstrate containerization using Docker.

- Docker is an open-source platform for developing, shipping, and running applications.
- It uses containerization technology to package applications with their dependencies.
- Ensures applications run **consistently across different environments** (development, testing, production).
- Provides **lightweight, portable, and scalable** solutions compared to traditional virtual machines.
- Widely used in **DevOps, cloud computing, and microservices** architecture.

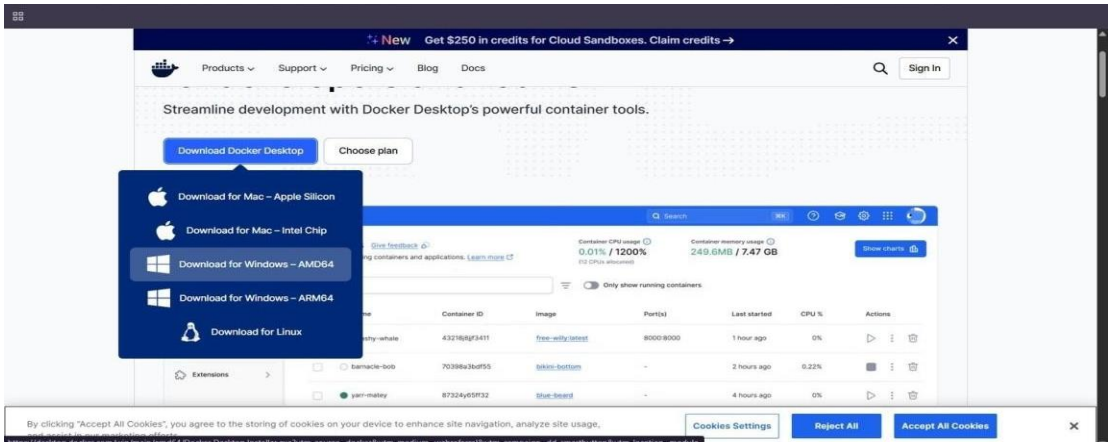


Steps to install Docker Desktop

Check System Requirements

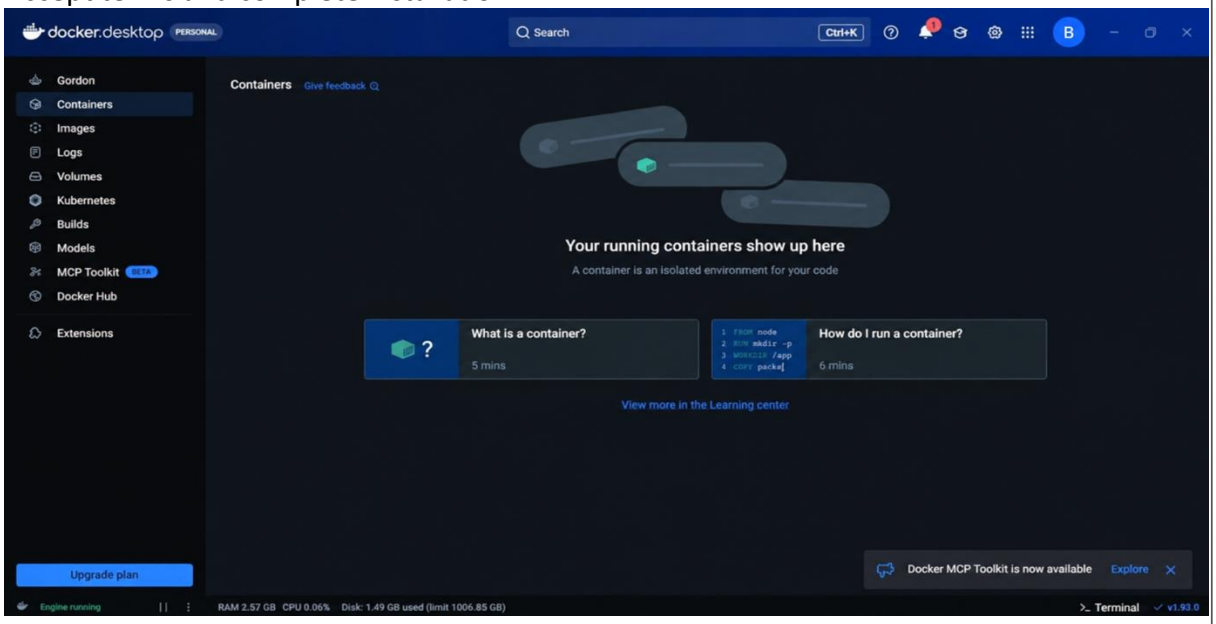
1. Windows: Windows 10/11 Pro, Enterprise, or Education (with WSL2 enabled)
2. macOS: macOS 10.15+
3. Linux: Ubuntu/Debian/Fedora/CentOS**Download Docker Desktop**

1. Visit https://www.docker.com/products/docker-desktop



Install Docker

- Run the installer and follow setup instructions.
- Accept terms and complete installation.



Verify Installation

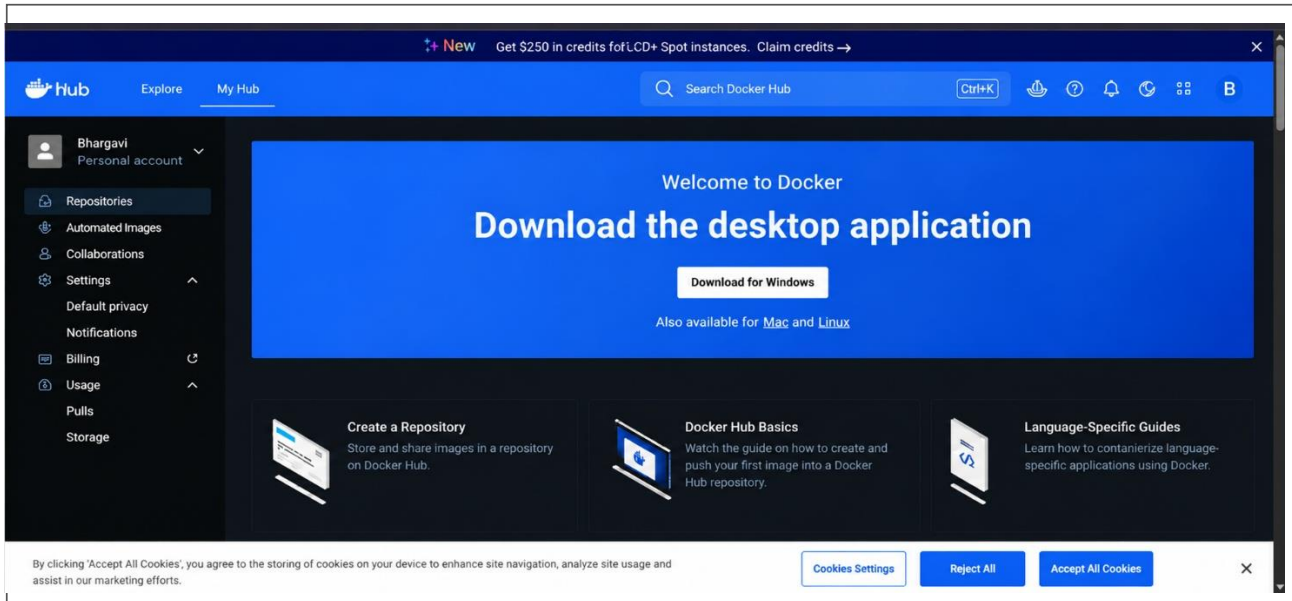
1. Open terminal/command prompt.
2. Run `docker --version`



Docker Hub

Docker Hub is Docker's official cloud-based registry service.

- It allows developers to:
  - **Store** Docker images.
  - **Share** images publicly or privately.
  - **Download** (pull) ready-to-use images for apps, databases, and tools.
- Acts like a GitHub for Docker images.
- Provides both:
  - **Official images** (verified by Docker, e.g., MySQL, Nginx, Python).
  - **Community images** (shared by developers).
- Create a Docker Hub Account.



Docker Image

A **Docker Image** is like a **blueprint** – it is a read-only template that contains the application code, dependencies, libraries, and configuration needed to run an application. You cannot run an image directly.

Docker Container

When you **start an image**, it becomes a **Docker Container**. A **Docker Container** is a running instance of an image – it is a lightweight, isolated environment where the application executes. You can create multiple containers from the same image, just like you can build multiple houses from the same blueprint.

Creating a Docker image

There are multiple ways to create the Docker images:

Using Dockerfile

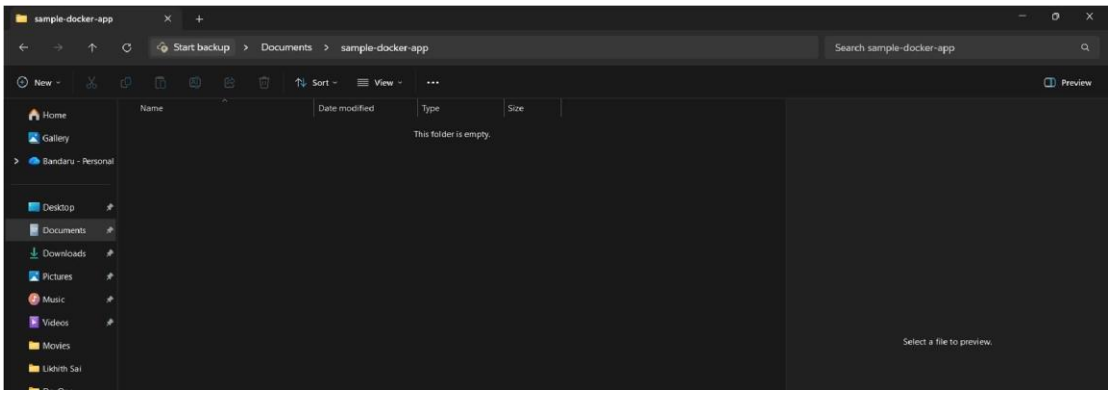
- Write a text file (Dockerfile) with instructions.
- Define base image, dependencies, environment, and commands.
- Steps to build the docker image from docker file and push to docker hub:

  Login to Docker Hub `docker login`

  Build the Docker Image from Dockerfile `docker build -t javaapp:1.0 .` **Tag the Image for Docker Hub** `docker tag javaapp:1.0 bhargavi1721/javaapp:1.0` **Push the Image to Docker Hub** `docker push bhargavi1721/javaapp:1.0`

Build the Docker Image from Dockerfile `docker build -t javaapp:1.0 .`

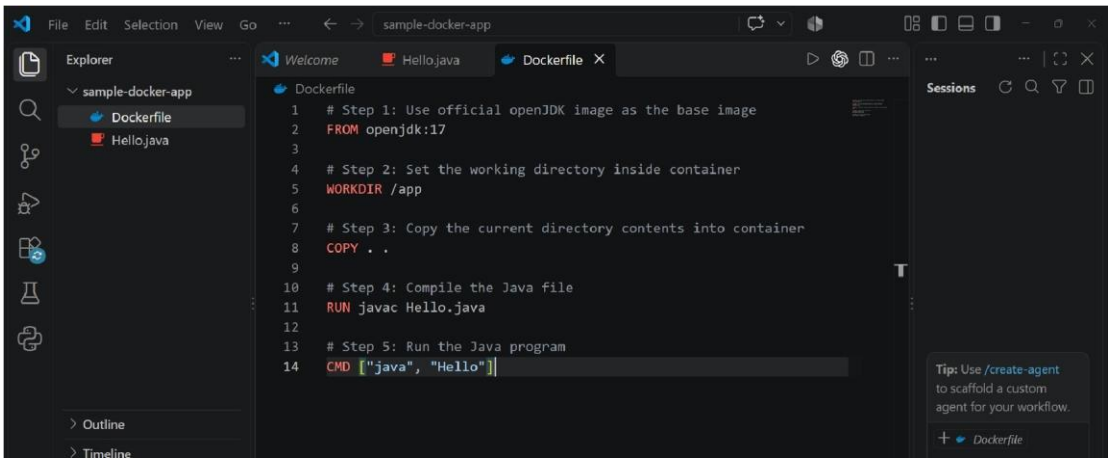
Make a folder and write a sample program and docker file in it.



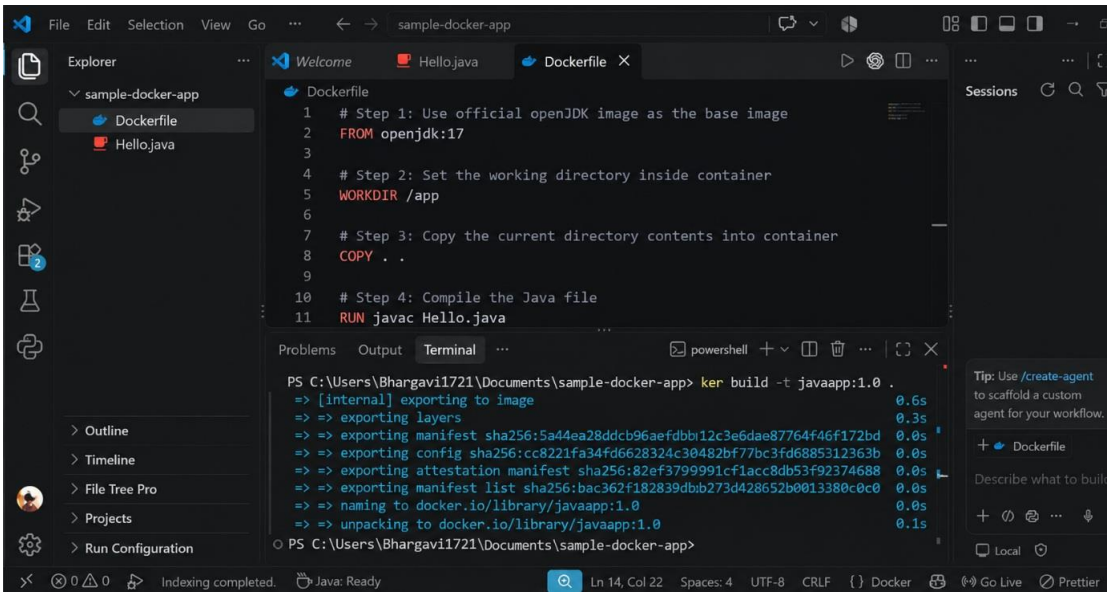
- Sample Java Program



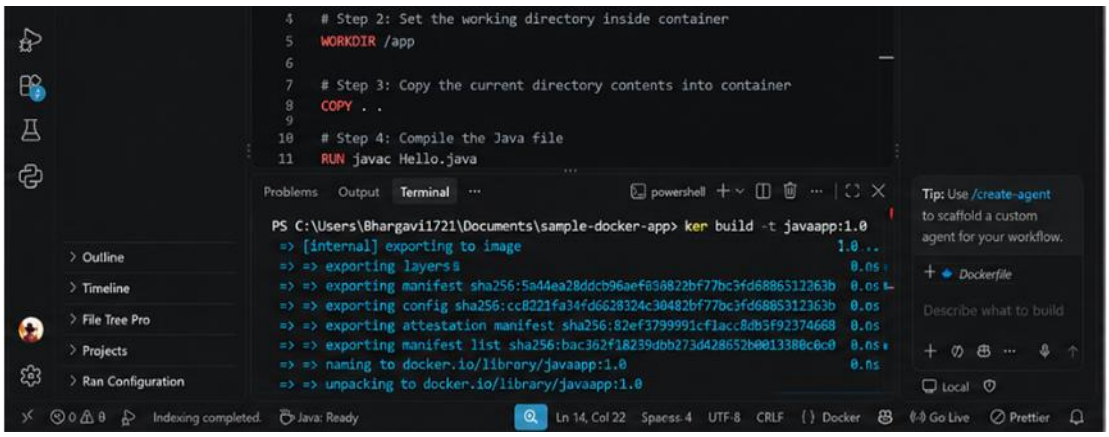
- Sample Docker File



- Build the docker image

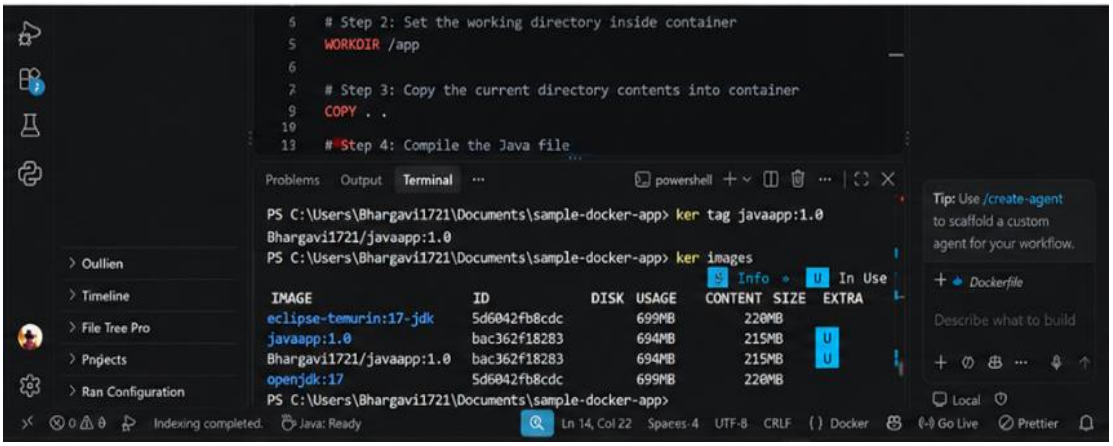


- Run it locally by command
  - docker run javaapp:1.0

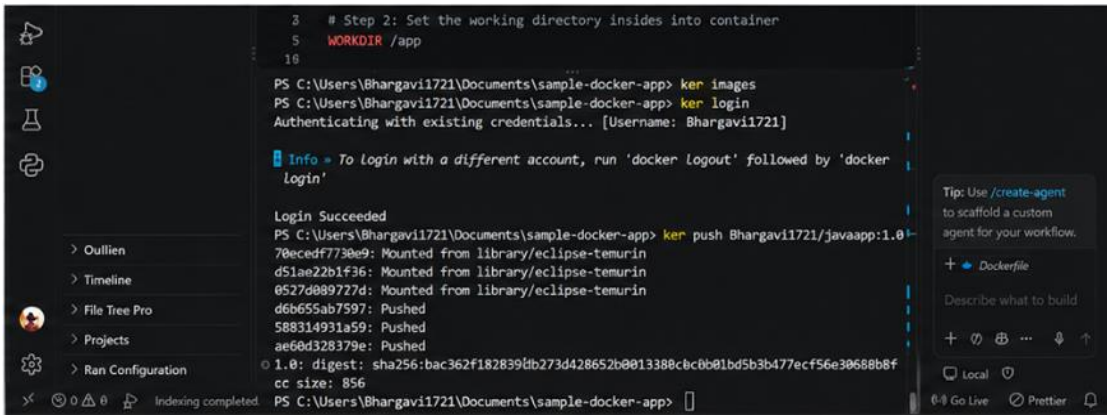


Tag the Image for Docker Hub `docker tag javaapp:1.0 bhargavi1721/javaapp:1.0`

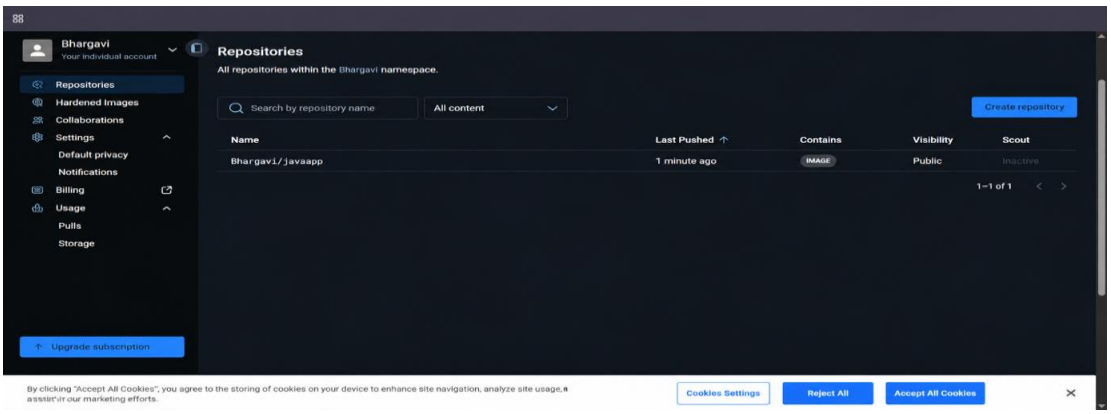
- Checking the docker images (docker images)



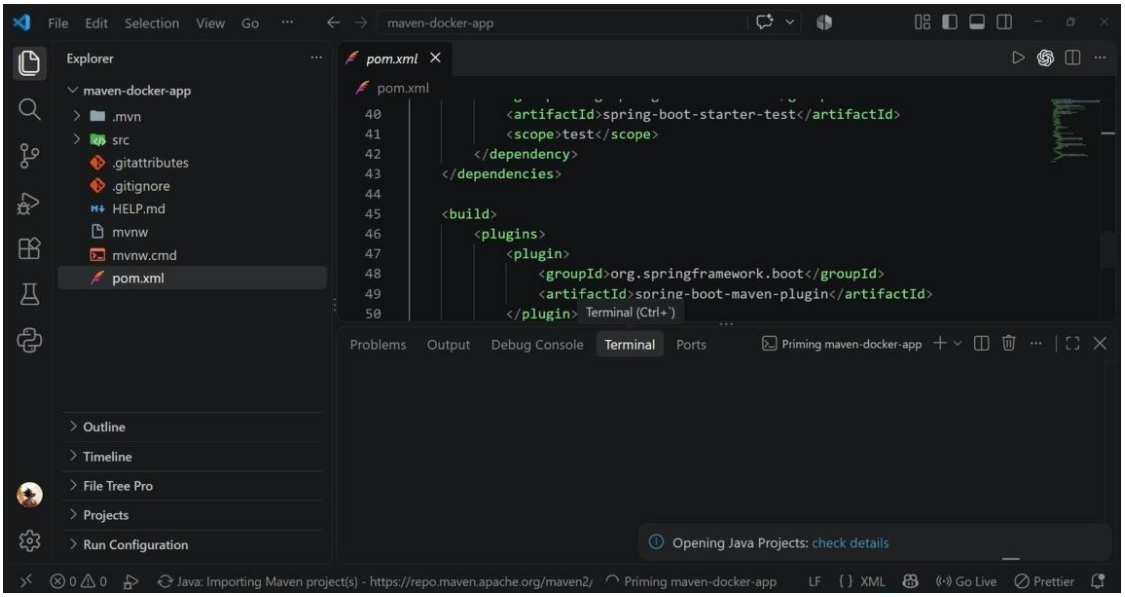
Login to Docker Hub `docker login`



Push the Image to Docker Hub `docker push bhargavi1721/javaapp:1.0`

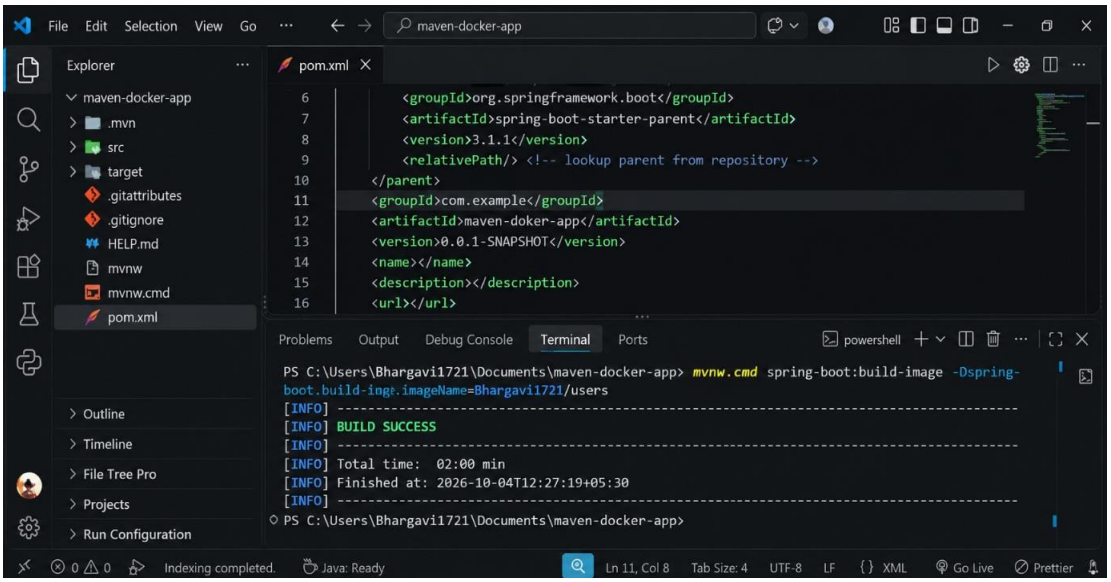


- Docker Hub After:

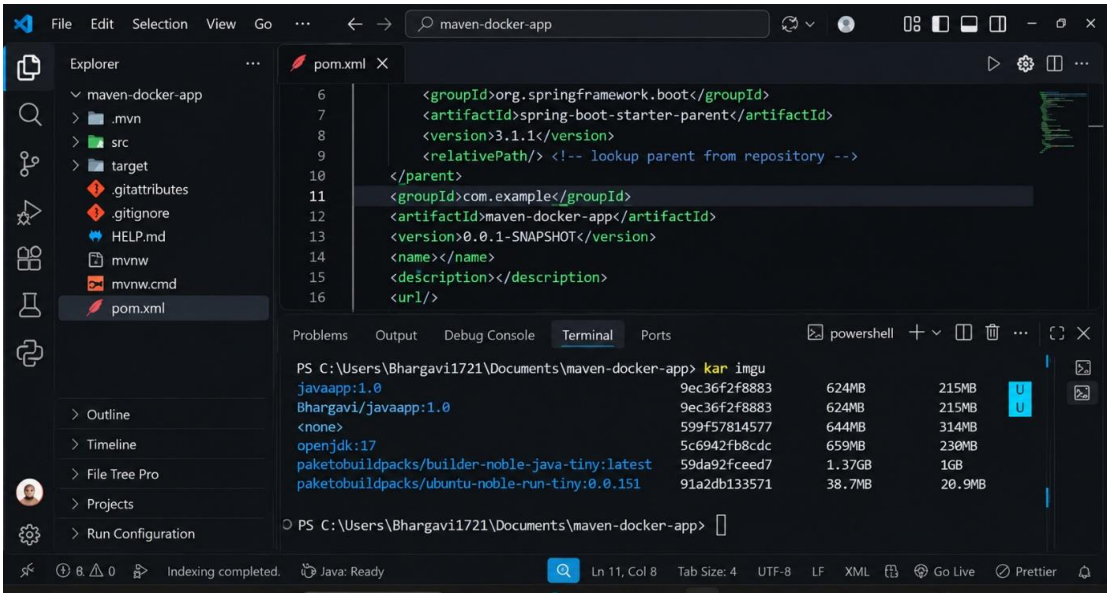


Using Maven Wrapper (for Java apps)

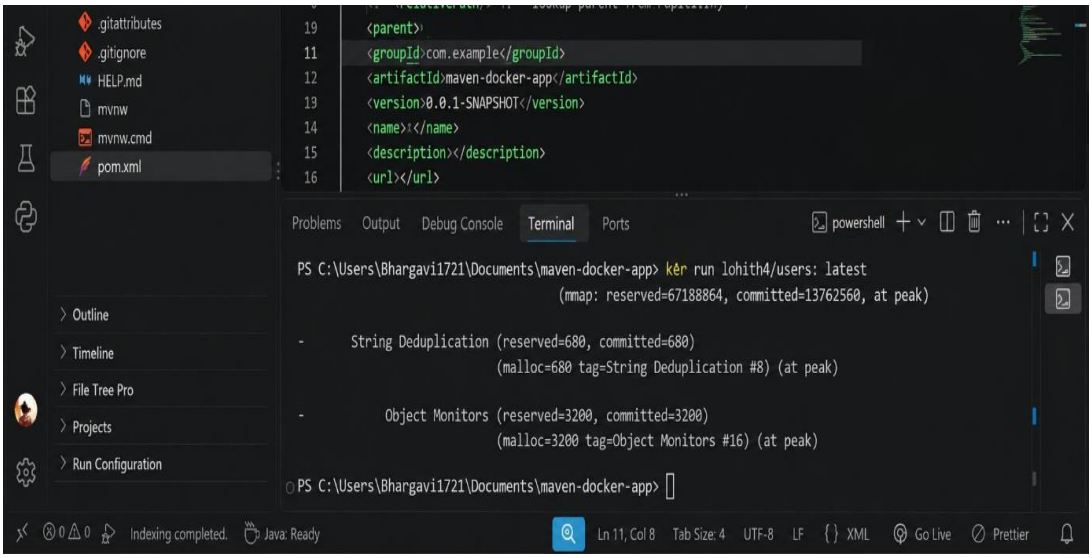
- Add Docker plugin in pom.xml.



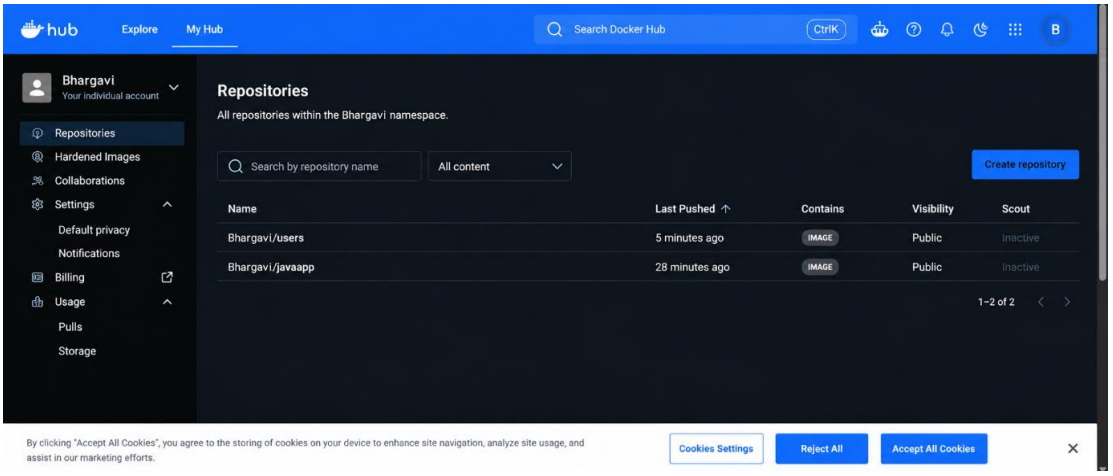
- Build Docker image as part of Maven build process.
- Run the example command `./mvnw.cmdspring-boot:build-image"-Dspring-boot.buildimage.imageName=bhargavi1721/users"`
- Here by using the above command docker image is directly built.



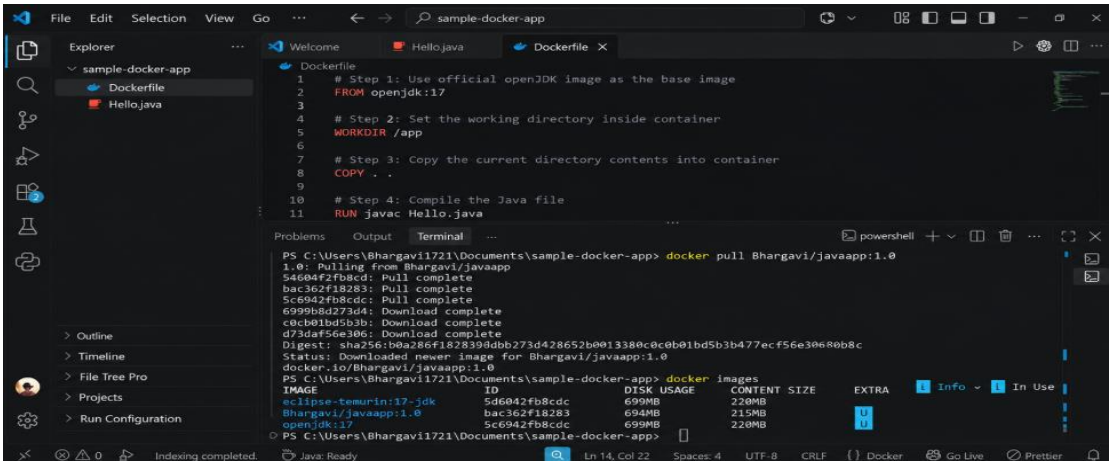
- The built image is available in the local docker images:



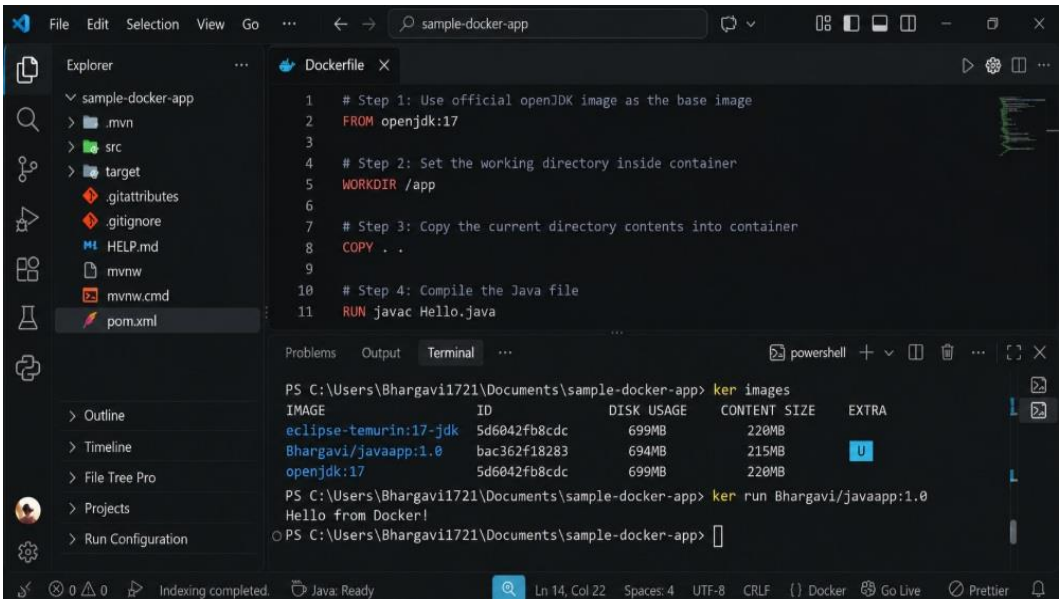
- And it is also pushed into docker hub:
  `docker push bhargavi1721/users:latest`



- Running the image:
  `docker run bhargavi1721/users:latest`

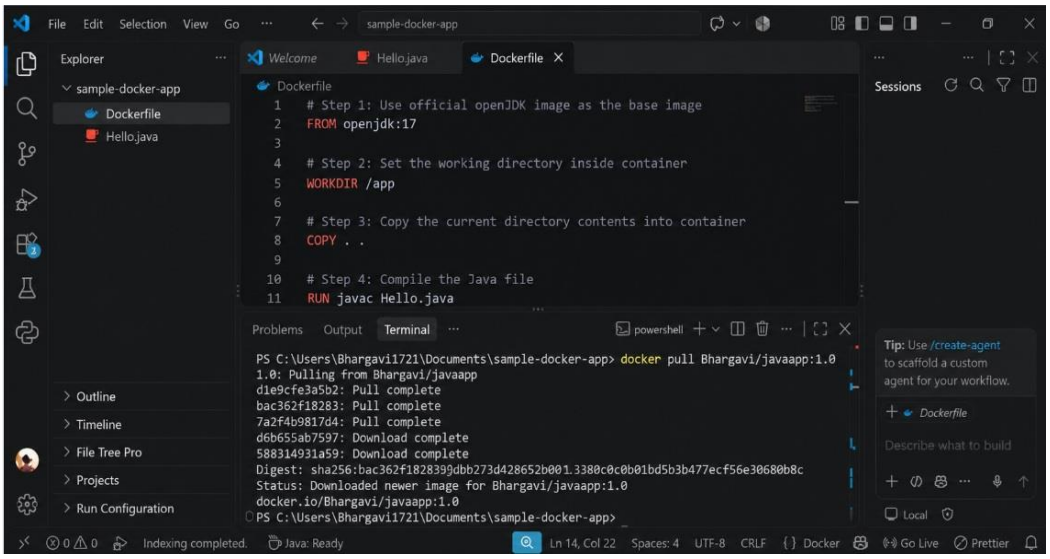


Sample Docker Images in Docker Hub



Pulling the Docker Image

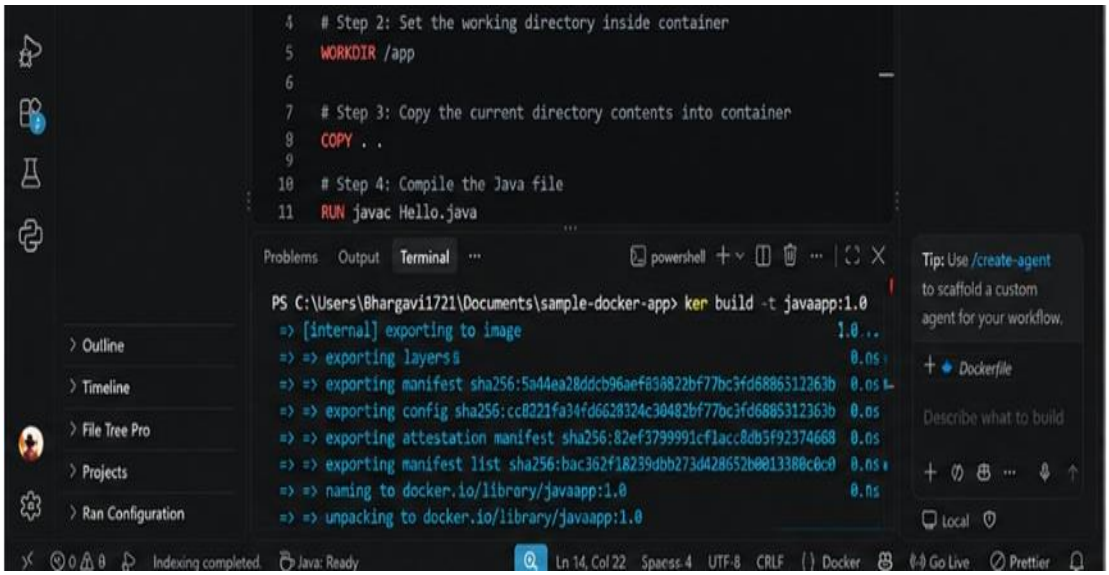
- Docker images are like blueprints – they need to exist on our local machine to run a container.
- If the image does not exist locally, we must pull it from docker hub.
- Example command to pull the docker image from the docker hub `docker pull bhargavi1721/javaapp:1.0`



Checking the Docker images

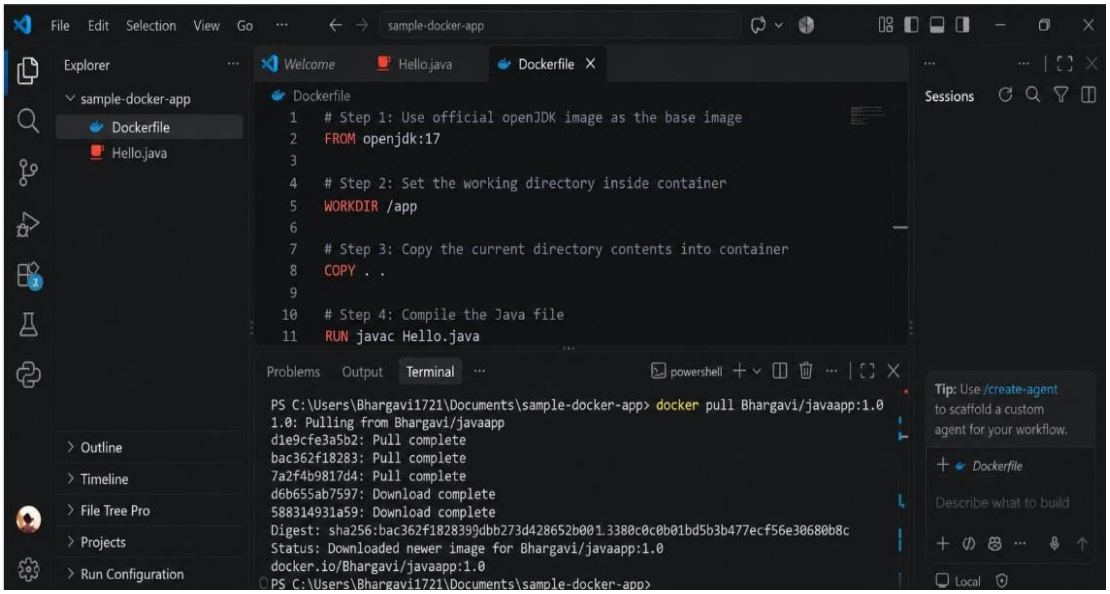
We can check the docker images by the command:

```
docker images
```



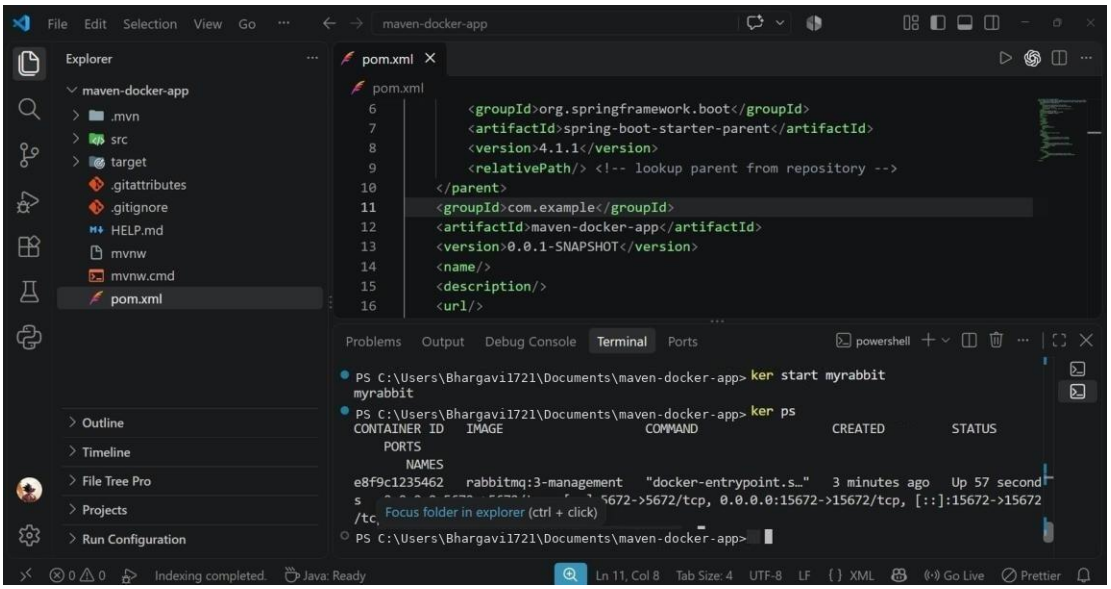
Running the image as Container

We can run the image as container by using the command: docker run [OPTIONS] IMAGE [COMMAND] [ARG...] `docker run bhargavi1721/javaapp:1.0`



Pulling the Public images

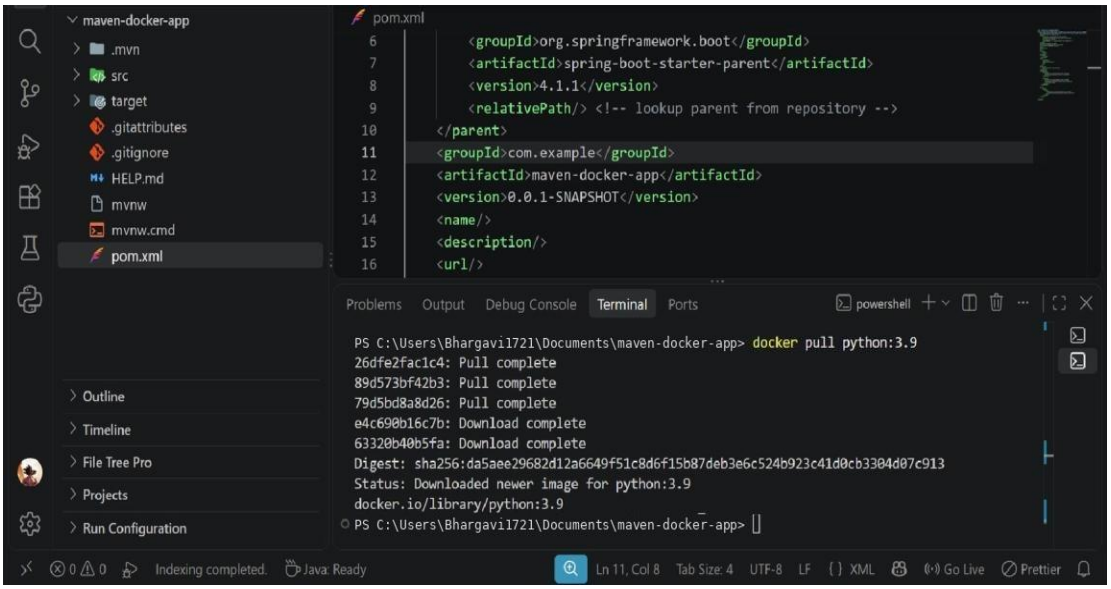
- We can also pull the public images that are available in docker and we can use them without installing the application.
  `docker pull rabbitmq:3-management`



- Running the container:
  `docker run -d --name myrabbit -p 5672:5672 -p 15672:15672 rabbitmq:3-management`
- After running the image, the RabbitMQ management page opens in the browser (port 15672):

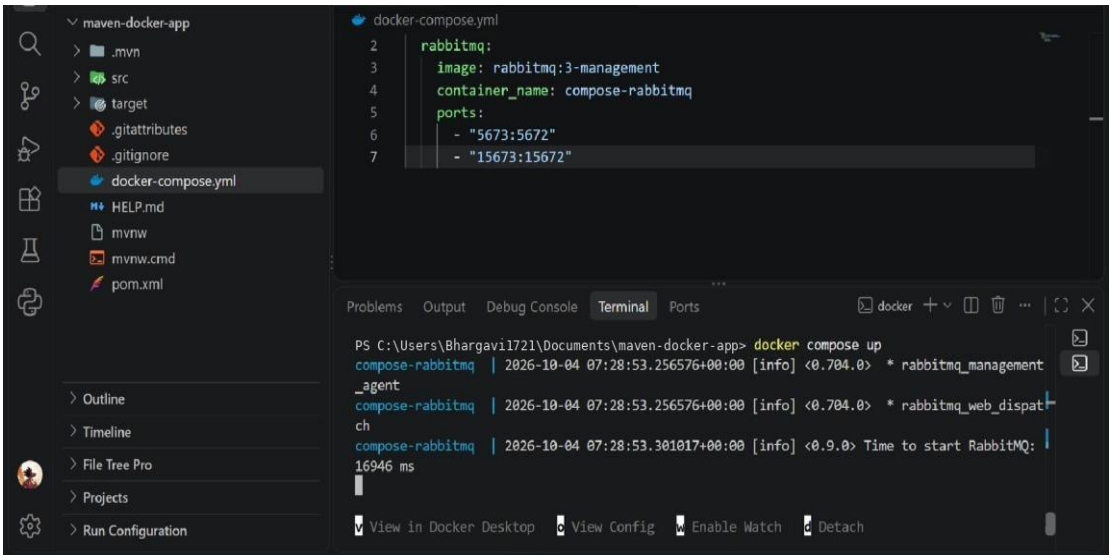


- A stopped container can be started again using docker start, and docker ps shows it running: `docker start myrabbit`



Pulling the Python Image From docker Hub `docker`

```
pull python:3.9
```

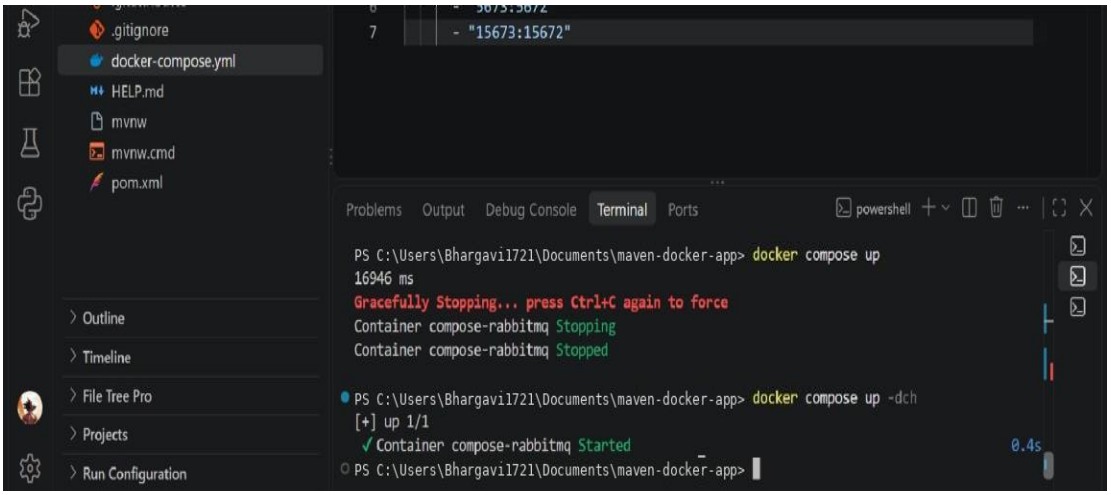


- **Docker Hub provides ready-made public images** for popular software like Python, MySQL, Nginx, RabbitMQ, etc.
- **No manual download or installation needed** – just pull and run in one command.
- **Auto-pull feature** – if the image is not on your system, Docker automatically fetches it from Docker Hub.
- **Run multiple versions easily** (e.g., Python 3.8, 3.9, 3.10) without affecting your system.
- **Consistency across machines** – the same image works on Windows, Linux, or Mac, avoiding "it works on my machine" issues.
- In short we can directly pull the image and use it without installing the application.

Run Containers Using Docker Compose

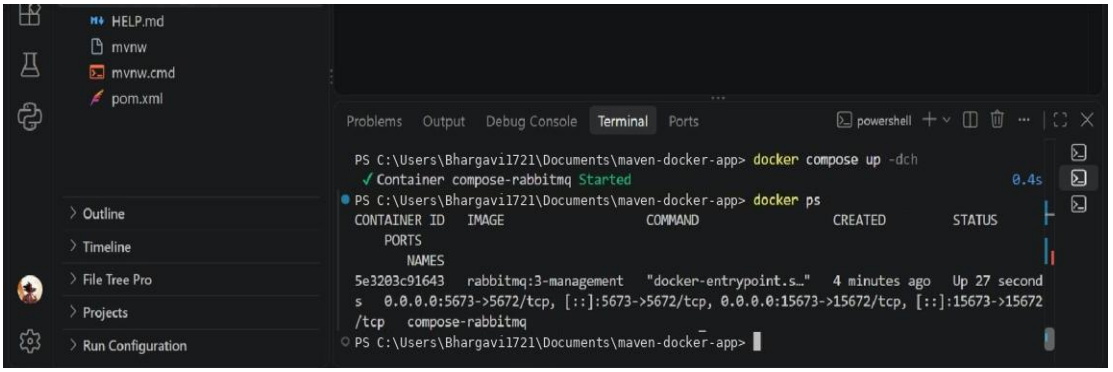
- Docker Compose lets you run single or multiple containers from images using a YAML file.

- Instead of running docker run manually for each container, you define **services** in the Compose file.
- Command to run docker compose file: `docker compose up`



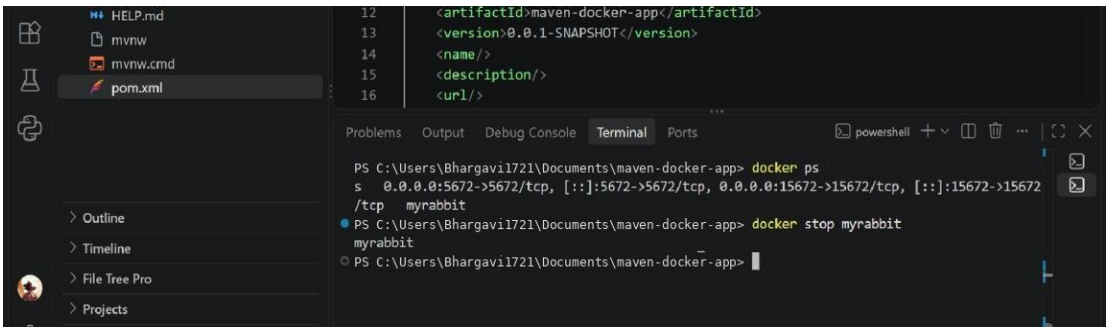
Running in detach Mode:

- Use **docker compose up -d** → Starts all services from docker-compose.yml in the background.
- Containers keep running even after you close the terminal → check them with docker ps.



Viewing the Running Containers:

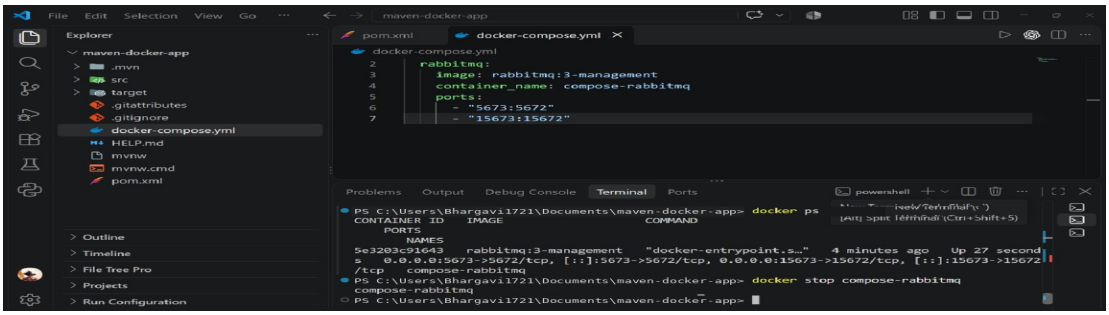
- To view the running containers we use the command `docker ps`

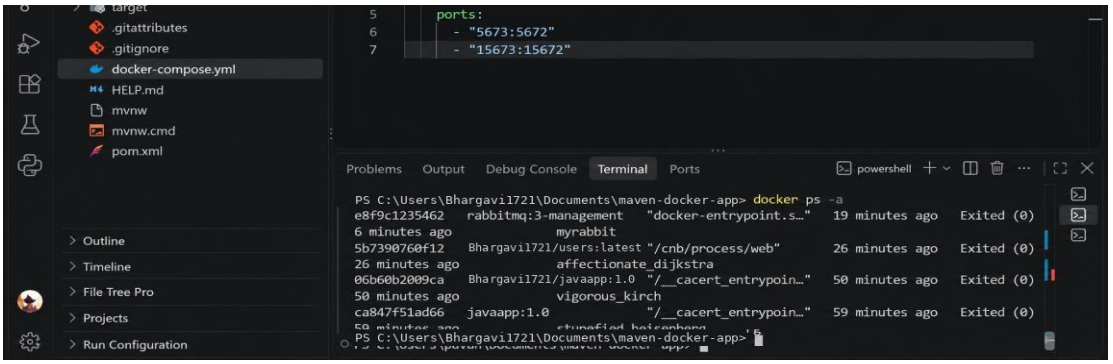


Stopping the Container

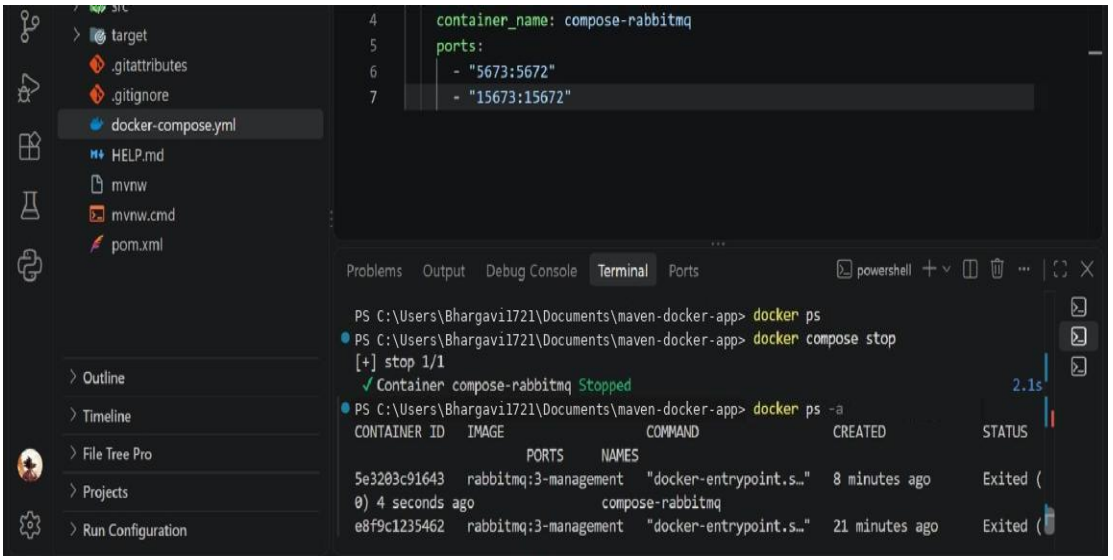
- To stop the running container we use the command `docker stop <container_id_or_name>`

Stopping the myrabbit container, and the RabbitMQ page can no longer be reached:



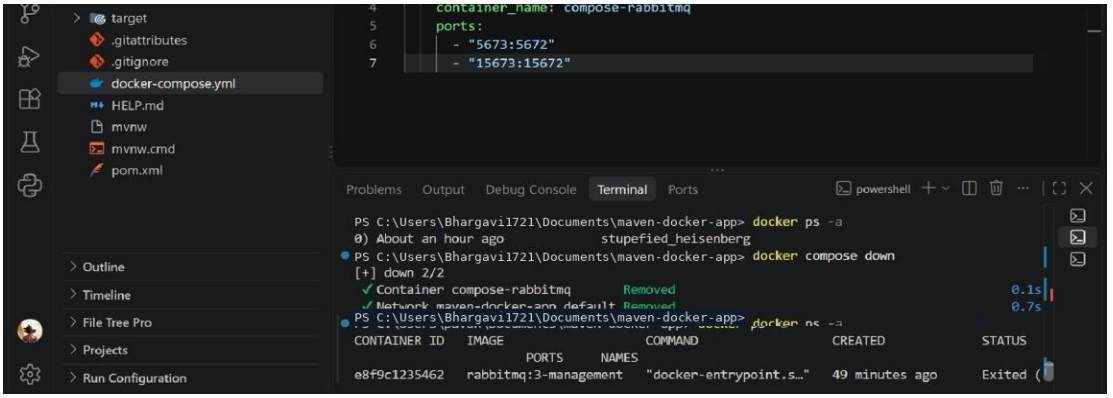


Stopping the compose container:



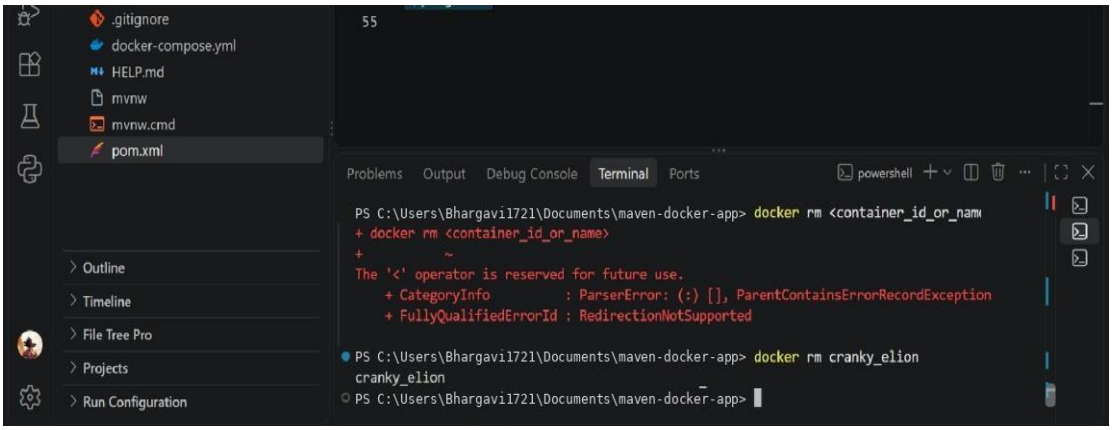
Checking all the containers Running as well as Stopped:

```
docker ps -a
```



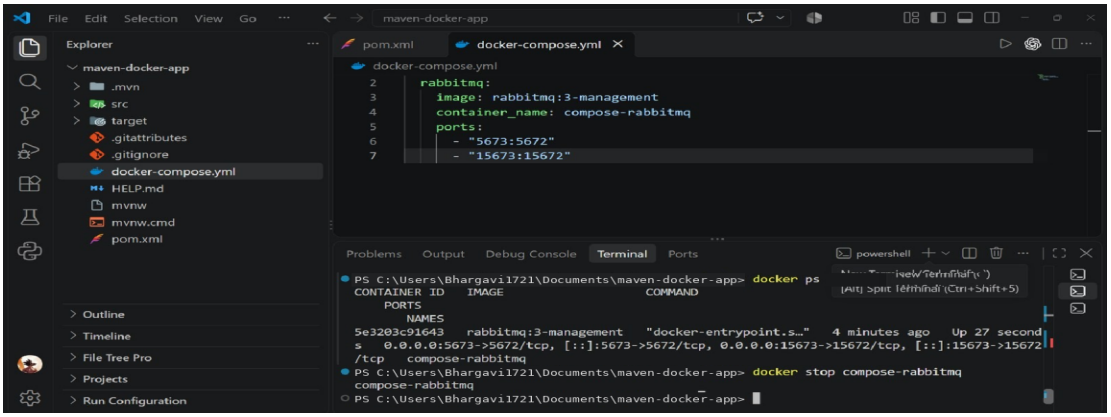
Stopping the Containers By Docker Compose File:

```
docker compose stop
```



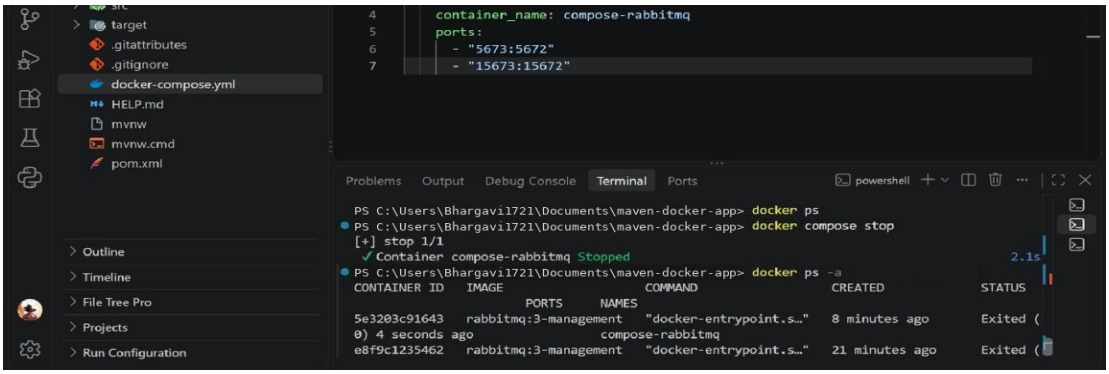
docker compose down stops and removes the containers (and the network) created by the compose file:

```
docker compose down
```



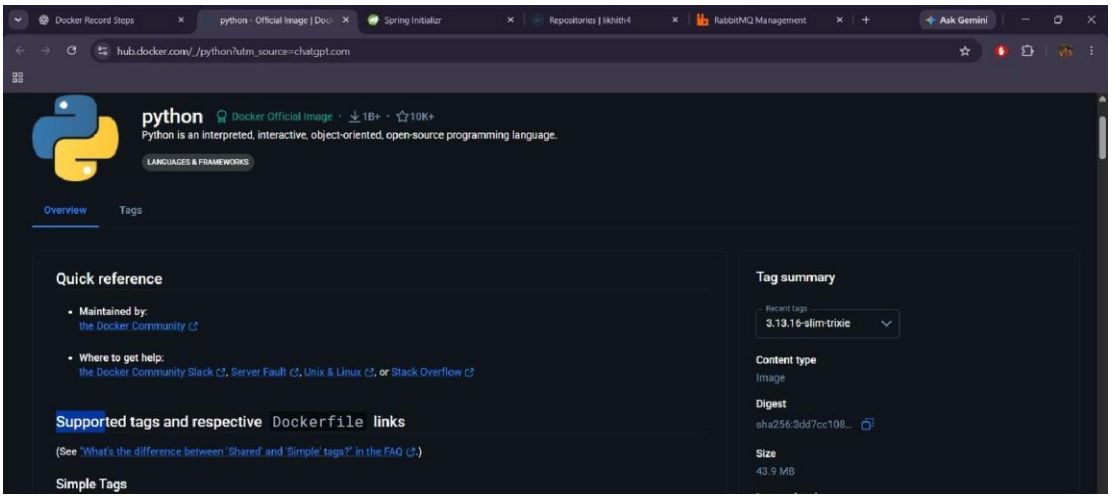
Removing a Container From Local System:

- To remove container we use the following command `docker rm <container_id_or_name>`
- Replace <container_id_or_name> with the actual container name or ID (for example, docker rm cranky_elion).
- Make sure that the docker desktop is opened in the background to run all the commands.

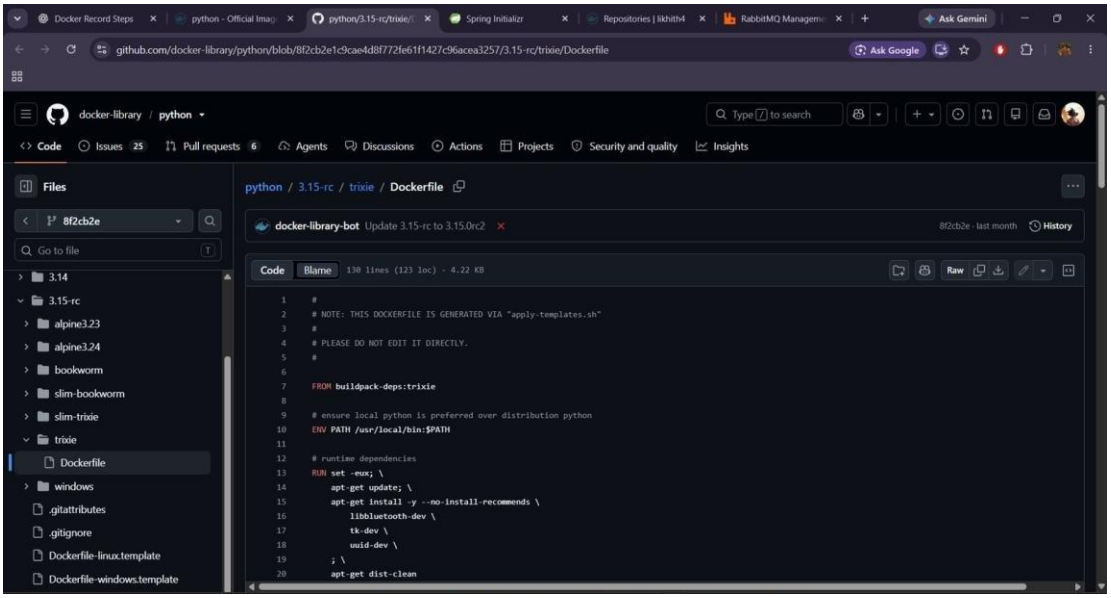


Viewing the docker file of public images

- Go to the docker hub.
- Click on any one image.



- Click on the links under Supported tags and respective Dockerfile links.
- We can view the docker files in the GitHub.



3. **Result:** Docker images were built and run locally, tagged and pushed to Docker Hub, public images were pulled, and containers were managed using Docker commands and Docker Compose.
4. **Conclusion:** The experiment demonstrated the Docker image-to-container workflow and the use of Docker Compose to start, inspect, stop, and remove containerized services.